Aqui está a explicação detalhada do seu código (**main.cpp**). Ele foi estruturado para atender aos requisitos da **PT4 (Exercício 7)** e da **Lista 43 (Exercício 2)**, combinando Ray Casting, simulação física e interação 3D.

1. **Configurações e Estruturas de Dados**
   Aqui definimos as regras do "universo" da simulação.
   - **Constantes Físicas**:
     - **“BOX_SIZE = 15.0f”**: Define o limite do mundo (de -15 a +15 em cada eixo
     - **“GRAVITY = 0.0f”**: Gravidade 0
     - **“DRAG = 0.99f”**: Simula a resistencia do ar/água. A cada quadro, a velocidade é multiplicada por 0.99, fazendo a bola parar suavemente.
   - **Struct “Sphere”**:
     - Armazena tudo sobre um objeto: Posição(**center**), Velocidade(**velocity**), Cor(**r,g,b**), Raio e se está se movendo(**isMoving**). Isso permite que cada bola tenha comportamento independente.
2. **Matemática Vetorial (Helpers)**
   O OpenGL padrão (C style) não tem uma classe de vetor pronta com operações matemáticas, então criamos funções manuais:
   - **add, sub, scale**: Para mover objetos e calcular distancias.
   - **dot**(Produto escalar): Essencial para calcular reflexos de luz e colisões (angulos de impacto)
   - **normalize**: Transforma qualquer vetor em tamanho 1 (apenas direção), usado para calcular a direção do tiro.

3. **O Coração da Seleção: Ray Casting(getRay)**
   Esta é a parte exigida no Exercício 2 da L43.
   - **Problema**: O mouse clica na tela 2D (X,Y), mas o mundo é 3D.
   - **Solução**(gluUnproject): A função projeta dois pontos no espaço 3D.
     1. Um ponto no plano “Near” (perto da camera, Z=0).
     2. Um ponto no plano “Far” (fundo do universo, Z=1).
     3. Subtraindo **End – Start**, criamos um vetor Direção (o raio) que sai do olho da camera e entra na cena.
4. **Intersecções (A Lógica de "Acertar" coisas)**
   O código possui dois tipos de cálculo de intersecção:
     1. Raio x Esfera (**raySphereIntersect**):
        - Usada para **Selecionar**.
        - Calcula a distância geométrica do centro da esfera até a linha do raio. Se a distância for menor que o raio da esfera, houve clique.
     2. Raio x Caixa (getBoxIntersection)
        - Usada para **Mirar**.
        - Calcula onde o raio do mouse bate nas paredes invisíveis da caixa (planos X, Y, Z).
        - Isso permite que você mire no teto ou no chão para fazer tabelas.
5. **O Motor de Física (Update)**
   Esta função é chamada repetidamente (aprox. 60 vezes por segundo) via glutTimerFunc. É aqui que a mágica acontece.
   - **Multitarefa**: O loop **for** percorre todas as esferas. Se **isMoving** for true, ele calcula a física. Isso permite lançar várias bolas simultaneamente.
   - **Movimento**: **Posição = Posição + Velocidade**.
   - **Colisão com Paredes (Boundary Check)**:
     - Verifica se **bola.x > 15** ou **bola.x < -15**.

        - Se bater, reposiciona a bola na borda (para não escapar) e inverte a velocidade (**vel.x *= -0.8**). O **-0.8** faz ela perder um pouco de energia (quicar menos forte).
    - **Colisão Esfera x Esfera**:
        - Verifica se a distância entre duas bolas é menor que a soma dos raios (**dist < r1 + r2**).
        - **Física de Reflexão:** Calcula o vetor "Normal" (direção da batida) e reflete o vetor de velocidade usando a fórmula de reflexão especular. Transfere parte da energia para a bola que estava parada, criando o efeito de sinuca.
6. Interação (**mouse** e **Estados**)
    1. **STATE_SELECT**:
        - O programa espera um clique.
        - Se o raio do mouse acertar uma bola (**raySphereIntersect**), mudamos para o estado de mira e pintamos a bola de amarelo.
    2. **STATE_AIM**:
        - O mouse agora controla a linha de mira amarela (**targetPos**).
        - Ao clicar novamente, calculamos o vetor **Alvo - CentroDaBola**, aplicamos uma força e definimos a velocidade.
        - **Importante**: Voltamos imediatamente para **STATE_SELECT**. Isso libera você para selecionar outra bola enquanto a primeira ainda viaja.
7. **Renderização (display)**

    Aqui desenhamos o que calculamos. Um detalhe crucial foi a ordem de desenho:
    1. **Câmera Orbital**: Usa coordenadas esféricas (**sin/cos** de **alpha** e **beta**) para converter os ângulos do mouse em uma posição X,Y,Z para o **gluLookAt**.
    2. **Esferas Primeiro:** Desenhamos as esferas sólidas.
    3. **Caixa Depois (drawBox)**: Desenhamos a caixa apenas como "arame" (**glutWireCube**).
        - *Por que essa ordem?* Se desenhássemos a caixa primeiro (mesmo sendo só linhas), em alguns drivers gráficos o buffer de profundidade poderia ocultar as esferas que estão "atrás" das linhas. Desenhar as esferas antes garante que elas sempre apareçam.

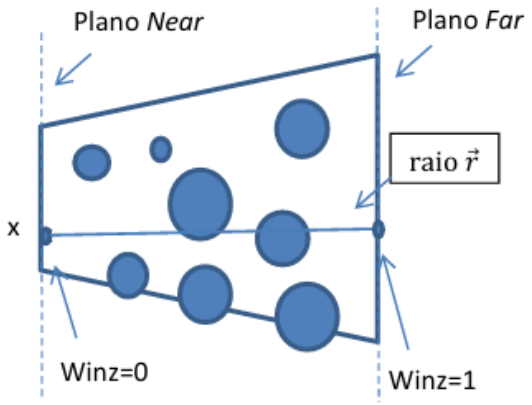
Resumo para sua Apresentação

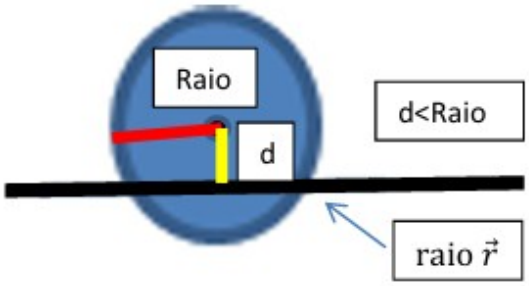
Se precisarem de um resumo técnico rápido:

> "O código implementa um integrador de física vetorial simples. A interação 2D-para-3D é feita via **gluUnProject** (Ray Casting). A detecção de colisão usa testes de intersecção raio-esfera para o mouse e testes de distância euclidiana para as colisões físicas, com resposta baseada na reflexão do vetor velocidade sobre a normal de contato."

Imagens para ajudar:

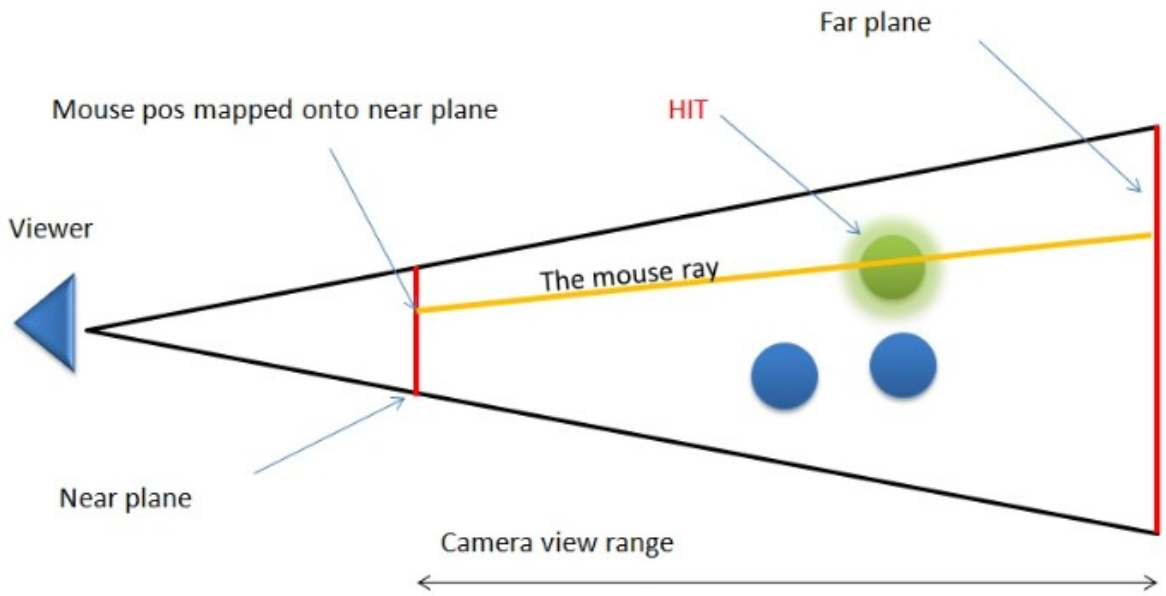
1) Clique na tela (x,y)
2) Determine o raio $\vec{r}$ no espaço que corresponde ao clique (x,y) usando o comando gluUnProject.
3) Realize a seleção através da determinação da colisão entre o raio $\vec{r}$ e as esferas. O teste de colisão será o seguinte: calcule a distância d do centro de cada esfera R até o raio $\vec{r}$ usando a distância ponto-reta. Se esta distância d for menor que o raio R da esfera, então a esfera passou o teste de colisão e será, por tanto, selecionada.





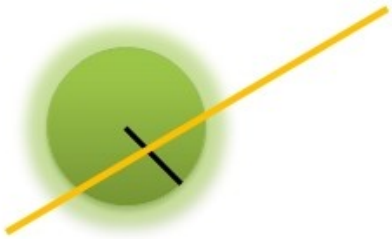
Idea

We would like to create a ray, starting in mouse position at the near plane. Then we will test what objects collide with the ray.



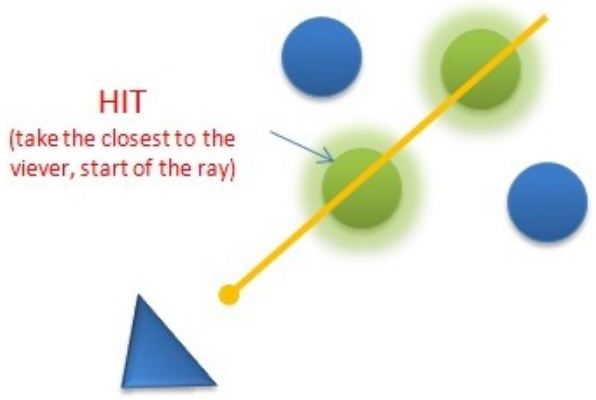
Collision between the ray and a sphere

What we have right now is the mouse ray and a bunch of objects in the scene that we want to check if there is a collision between them and the ray.



Dist(ray, sphere_center) < sphere_radius

As you can see in the picture the mouse ray hits yellow ball that is in the scene. Of course testing collision between the ray and a sphere is one of the simplest cases. It is a good starting point for more advanced collision tests: with **AABB** (Axis Aligned Bounding Boxes), **OBB** (Oriented Bounding Boxes) or even arbitrary meshes. On the other hand usually we have "hierarchy" of collision tests: from simple to more advanced… so testing with spheres is usually a must-have tool.



blablabla